Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Kod kusurları ve refactoring'e dönüş
  - Geçen dersin tanımları kısaca tekrarlandı: refactoring, bir kodun dış arayüzünü değiştirmeden iç çalışma yapısını değiştirmektir; amaç daha iyi iç kalite özellikleri ve gelen gereksinimlere karşı daha esnek çözümlerdir.
  - Kod kusuru (code smell) hatalı kod değildir: bug yoktur, program doğru çalışır ve ihtiyacı karşılar. Ama kullanılan dil ya da yaklaşımın önerilerine aykırı yapılmış şeyler ya da tasarım kusurları vardır ve bunlar yakında gelecek değişikliklerde sorun çıkarır.
  - İçerik Martin Fowler'ın "Refactoring" kitabından düzenlendi. Kitaptaki bütün kod kusurları işlenmiyor. Kod kusurları ile refactoring eylemleri arasında birebir değil M×N ilişki vardır; hoca bu ilişkide en çok iç içe geçmiş olanları seçti.
  - Kod kusurları nispeten küçük ölçekli kusurlardır. Daha büyük ölçekli kusurlar "anti-patterns" konusudur.
  - Kod örnekleri "önce" ve "sonra" diye yan yana gösterilir. Alt çizgiyle başlayan değişkenler üye alandır; bu, siyah beyaz malzemede IDE renklendirmesi olmadan üye alanı ayırt etmek içindir (ekrandan, 00:17:50).
- Duplicated Code (yinelenen kod) (ekrandan, 00:17:50)
  - Aynı kodun ya da çok benzer kod yapısının birden fazla yerde tekrarlanmasıdır.
  - İşlevsellikte değişiklik gerektiğinde (hata düzeltme ya da gereksinim değişikliği) tekrarlanan her yerde aynı işlem yapılmak zorunda kalınır; sonuç bir bakım kabusudur.
  - Çözüm: Extract Method (yinelenen kodu bir metot altında toplamak), Pull Up Method, Template Method kalıbı ya da metodu ilgili sınıflardan birine taşımak.
- Extract Method (metot çıkartma) (ekrandan, 00:20:00, 00:30:00)
  - Bir metot içindeki kodun bir kısmını yeni bir metot olarak belirlemektir. Birçok kod kusurunu düzeltir.
  - Yeni metoda amacını açıkça belirten isabetli bir ad verilir; böylece alt düzey ayrıntılarla uğraşmadan kodun ana hatları anlaşılır.
  - Birinci örnek: `printOwing(double amount)` içindeki ayrıntı yazdırma satırları `printDetails(double amount)` metoduna çıkarılır; eski metot yeni metodu çağırır.
  - İkinci örnek: `printOwing()` içinde üç yorumla ayrılmış üç iş vardır (banner yazdırma, borç hesaplama, ayrıntı yazdırma). Her biri ayrı metoda çıkarılır: `printBanner()`, `getOutstanding()`, `printDetails(double outstanding)`. `printOwing()` yerinde kalır ve `printBanner(); printDetails(getOutstanding());` olur. Döngü `while/Enumeration` yerine for-each ile yazılarak bir satır daha kısaldı.
  - Dışa açık `printOwing()` public kalır; çıkarılan metotların public olması gerekmez, başka yerden çağrılmayacaksa private bile olabilir.
  - Kazanımlar: metot kısalır ve anlaşılır olur; yorum satırına gerek kalmaz, çünkü metot adı ne yapıldığını anlatır; odaklanmış kod yeniden kullanılabilir; uyum yükselir.
- Long Method (aşırı uzun kod) (ekrandan, 00:25:20)
  - Metodun uzunluğu arttıkça anlaşılması zorlaşır, değişikliklerden etkilenme olasılığı artar, birden fazla ilgi alanını kapsama olasılığı artar (uyum düşer).
  - Çözüm: Extract Method (uyumlu komutlar seçilerek bir metot altında toplanır); ayrıca Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
  - Hocanın örneği: 20 satırlık tek metot yerine 8'er satırlık üç metot. Toplam satır 24'e

çıkar, ama bir sorunu anlamak için yalnız 8 satıra odaklanılır ve her metot kendi içinde daha uyumlu olur.

- Pull Up Method (metodu yukarı çekme) (ekrandan, 00:40:00, 00:42:30)
  - Kardeş (ya da kuzen) sınıflarda yinelenen metotlar üst sınıfa çekilerek tek yerde kodlanır. "Pull up/pull down" kalıtım hiyerarşisinde yukarı çıkarmak ve aşağı indirmek demektir.
  - Örnek: `Salesman` ve `Engineer` sınıflarındaki aynı `getName` metodu `Employee` üst sınıfına çekilir.
  - Özel durum: soyut `Customer` (`lastBillDate`, `addBill(dat: Date, amount: double)`) altında `Regular Customer` ve `Preferred Customer` var. İkisinde de `createBill(Date)` ve `chargeFor(start: Date, end: Date)` var; `chargeFor` çok farklı kodlanmış, `createBill` ise çok benzer. `createBill(Date)` üst sınıfa çekilir, `chargeFor` üst sınıfta soyut tanımlanır, alt sınıflarda farklı kalır.
  - Daha karmaşık durumda Template Method kalıbı kullanılır: tekrarlanan kısımlar üst sınıftaki template metotta, farklılaşan kısımlar alt sınıflarda gerçeklenen ve üst sınıfta soyut tanımlanan metotlarda olur.
  - Tekrarlanan kod kalıtım ilişkisi olmayan sınıflardaysa tasarım büyük olasılıkla baştan yanlıştır (sorumluluklar doğru atanmamıştır). Yeniden tasarıma zaman yoksa ya da geriye uyumluluk çok önemliyse, çıkarılan metot bu sınıflardan birine taşınır ve diğerleri onu çağırır; belki yeni bir sınıf da oluşturulabilir.
- Decompose Conditional (karar verme komutunu parçalama) (ekrandan, 00:50:00)
  - Aşırı karmaşık `if` ya da `while` koşulları daha küçük alt koşullara bölünür. Anlaşılabilirlik artar, hata yapma olasılığı azalır ve yapılan hata daha kolay bulunur.
  - Önce: `if (date.before(SUMMER_START) || date.after(SUMMER_END)) charge = quantity * _winterRate + _winterServiceCharge; else charge = quantity * _summerRate;`
  - Sonra: `if (notSummer(date)) charge = winterCharge(quantity); else charge = summerCharge(quantity);` ve üç private metot: `notSummer(Date)`, `summerCharge(int)`, `winterCharge(int)`.
  - Hoca zaman çizgisi üzerinde koşulun doğru olduğunu gösterdi: yaz başlangıcından önce ya da yaz bitişinden sonra olan tarihler kıştır.
  - Hocanın önerisi: düz mantık daha kolay anlaşılır; `notSummer` yerine `isSummer` metodu kodlanıp `if (isSummer(date)) ... else ...` yazılabilir. Bu da dış arayüzü değiştirmediği için refactoring'dir.
- Long Parameter List (uzun parametre listesi) (ekrandan, 01:06:00)
  - Bir metodun parametre sayısının fazla olması, hatta bu parametrelerin bazılarının başka metotlarda da birlikte geçmesidir. Bu parametreler birlikte bir varlığın durum bilgisinin bir kısmını ya da tamamını oluşturur.
  - "Fazla"nın sınırı kolay değildir; belirli kalitenin üstündeki yazılımlar yazılım metrikleriyle incelenip olması gereken değer aralığı belirlenebilir.
  - Çözüm: Introduce Parameter Object ve Preserve Whole Object.
- Introduce Parameter Object (parametre nesnesi oluşturma) (ekrandan, 01:10:00)
  - Uygun parametreler üye alan olarak yeni bir sınıfta toplanır ve o türden bir nesne parametre olarak kullanılır.
  - Örnek: `Customer` sınıfında `amountInvoicedIn`, `amountReceivedIn`, `amountOverdueIn` metotlarının hepsi `(start: Date, end: Date)` alır. Bu ikisi birlikte bir zaman dilimidir; `DateRange` sınıfı oluşturulur ve metotlar `(DateRange)` alır.
  - Kazanç yalnız parametre listesini kısaltmak değildir: `DateRange` yalnız iki veriyi birleştiren kayıt yapısı olarak kalmaz, `isWithinRange(Date)` gibi metotları da içerir; böylece Decompose Conditional ya da Extract Method da yapılmış olur.
  - Bu eylemde `Customer` metotlarının imzası değişir, yani sınıf düzeyinde refactoring ilkesi ihlal edilir. Hoca bunu kabul etti; ama sistem düzeyinde bakınca tümleştirme ve sistem testleri değişmez, yalnız birim testleri değişir. Kazanımlar daha fazla olduğu için hoca bu eylemi önerdi.
- Preserve Whole Object (tüm nesneyi gönder) (ekrandan, 01:20:00)
  - Bir nesnenin çeşitli üyelerini aynı metodun parametreleri olarak göndermek yerine nesnenin kendisi gönderilir.
  - Önce: `int low = daysTempRange().getLow(); int high = daysTempRange().getHigh(); withinPlan = plan.withinRange(low, high);`. Sonra: `withinPlan = plan.withinRange(daysTempR` ve `withinRange` metodu gerektiği gibi değiştirilir.

  - Zayıf yönü: bağımlılık artabilir; parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bir bağımlılık (UML'de kesikli çizgili kullanma ilişkisi) oluşur. Bu çok sakıncalıysa eylem yapılmaz.
  - Başka bir sakınca: iki `int` yerine bütün nesneyi aktarmak daha fazla veri demektir. Yerelde ihmal edilebilir, ama iki nesne farklı bilgi sistemlerinde (örneğin bankalarda) ya da ağ üzerinden haberleşiyorsa süre ve masraf artabilir. Böyle bir sakınca yoksa eylem önerilir; ileride gereken üye sayısı artabileceği için parametre listesi de uzamaz.
- Replace Temp with Query (geçici değişken yerine erişim metodu kullanımı) (ekrandan, 01:24:30, 01:30:00, 01:31:30)
  - Bir ifadeyi geçici değişkende saklayıp kullanmak yerine ifadeden metot(lar) çıkarılır.
  - Geçici değişkenler çalışma anı başarımı için kullanılır (hesap bir kez yapılır, sonuç birçok yerde kullanılır). Ama metodun uzamasını ve karmaşıklaşmasını teşvik eder.
  - Örnek: `getPrice()` içinde `basePrice = _quantity * _itemPrice`, `basePrice > 1000` ise `discountFactor = 0.95`, değilse `0.98`, sonuç `basePrice * discountFactor`. Sonra: `getPrice() { return basePrice() * discountFactor(); }`, private `basePrice()` ve `discountFactor()` metotları.
  - Avantaj: kopyala-yapıştır kodlamayı ortaya çıkmadan azaltır. Benzer işlem gerektiğinde kopyalamak yerine sorgu metodu çağrılır; hata bulunursa yalnız sorgu metodunda düzeltilir.
  - Tartışma: başarımı düşürür, çünkü fazladan metot çağırma yükü gelir. `getPrice` örneğinde `basePrice()` iki kez çağrılır. Şerh: modern derleyiciler bu ek yükü ortadan kaldırmış ya da önemsizleştirmiştir.
  - Döngüde etkisi: `printRecipt()` örneğinde `thisAmount` yerine iki yerde `item.getCharge()` çağrılır; birikimci `totalAmount` geçici değişkeni kalmak zorundadır. 5 öğe için çağrı sayısı 5'ten 10'a çıkar. Kayıt sayısı büyükse bu hissedilebilir; profiler ile öncesi ve sonrası ölçülür, ek yük kabul edilemezse eylem yapılmaz ya da geri alınır.
- Replace Method with Method Object (metot yerine yeni türden nesne kullanımı) (ekrandan, 01:36:50)
  - Metot o kadar uzun ve geçici yerel değişkenleri o kadar iç içe kullanıyor ki Extract Method yapılamıyorsa kullanılır.
  - Yeni bir sınıf oluşturulur, bütün metot o sınıfa taşınır, yerel değişkenler o sınıfın üyeleri olur. Ardından bilinen düzenlemeler daha rahat yapılır; en azından sorun izole edilmiş olur. Yeni sınıfa taşınan metodun adına benzer bir ad verilir.
  - Örnek: `Order.price()` içinde `primaryBasePrice`, `secondaryBasePrice`, `tertiaryBasePrice` ve uzun bir hesap var. `PriceCalculator` sınıfı oluşturulur (bu üç üye alan ve `compute()`); `Order.price()` yalnız `return new PriceCalculator(this).compute();` der. `Order` dış arayüzü değişmez; iş orada yapılmaz, oradan yaptırılır.
- Large Class (aşırı büyük sınıf) (ekrandan, 01:40:00)
  - Sınıfın büyük olması sınıfa ilgisiz sorumluluklar atandığının göstergesidir; uyum düşer. Üye alan sayısının fazlalığı da bir göstergedir.
  - Metotlar kısa olsa bile çok sayıda olabilir; büyüklük yalnız uzun metotlardan kaynaklanmaz.
  - Çözüm: Extract Class, Extract Subclass, Duplicate Observed Data.
- Kaynak kod görselleştirme ve metrikler (ekrandan, 01:43:30, 01:48:10)
  - Hoca Lanza, Marinescu ve Ducasse'nin "Object-Oriented Metrics in Practice" kitabını gösterdi. "System Complexity" görünümünde her dörtgen bir sınıftır: genişlik üye alan sayısı, yükseklik metot sayısı, renk koyuluğu kod satırı sayısıyla orantılıdır. Refactoring'e nereden başlanacağına karar vermede yararlıdır.
  - Çok metot kötü değildir; parçalara ayırma metot sayısını artırır. Metot sayısı az ama koyu (çok satırlı) bir sınıfta uzun metot olup olmadığına bakılmalıdır.
  - Eşik değerler dile göre değişir. Kitap 45 Java ve 37 C++ sistemi inceleyip WMC, AMW, LOC/Class, NOM/Class için düşük, ortalama, yüksek ve çok yüksek eşikler türetmiştir (tablo aşağıda, Detaylı Açıklamalar).
- Extract Class (sınıf çıkartma) (ekrandan, 01:52:30)
  - Birbiriyle ilgisiz iki iş yapan bir sınıfı bu işlere göre ikiye ayırmaktır; ikinci işle ilgili üyeler seçilerek yeni sınıfa taşınır. Birçok kod kusurunu düzeltir.
  - Örnek: `Person (name, officeAreaCode, officeNumber, getTelephoneNumber)` → `Person (name, getTelephoneNumber)` ve `officeTelephone` ilişkisiyle (çokluk 1) `TelephoneNumber (areaCode, number, getTelephoneNumber)`. `Person.getTelephoneNumber` yalnız

köprü (vekil) olur.
  - Yeni telefon türleri gerektiğinde her biri için iki üye alan yerine tek üye alan eklenir.
  - Bağlantı: ilk haftalardaki uyum ölçümünde üye alan-metot erişim grafiği bölünerek iki sınıf oluşturulmuş ve uyum iyileşmişti; Extract Class bunu yapar.
- Extract Subclass (alt sınıf çıkartma) (ekrandan, 01:54:30)
  - Bir sınıfın bazı üyeleri her zaman kullanılmıyorsa bu üyelerden yeni bir alt sınıf oluşturulur.
  - Örnek: `Job Item` (`getTotalPrice`, `getUnitPrice`, `getEmployee`). Bazı işler çalışana bağlıdır, bazıları değildir. `Labor Item` alt sınıfı (`getUnitPrice`, `getEmployee`) oluşturulur. Her `Labor Item` bir `Job Item`'dır ama her `Job Item` bir `Labor Item` değildir; kalıtım özelleşme-genelleşme ilişkisidir.
- Duplicate Observed Data (gözlenen verinin çoğullanması) (ekrandan, 02:00:00)
  - Uygulama alanıyla ilgili bir verinin yalnız bir GUI sınıfında saklanması doğru değildir. Veri, uygulama alanı/bilgi düzeyi katmanında yeni bir sınıfa taşınmalı ve GUI nesnesi ile yeni bilgi nesnesi arasında Observer kalıbı kurulmalıdır.
  - Örnek: `Interval Window` hem `startField`, `endField`, `lengthField` (TextField) ve `..._FocusLost` GUI olaylarını hem de `calculateLength`, `calculateEnd` iş mantığını içerir; katmanlaşma bozulmuştur. Sonra: `Interval Window Observer` arayüzünü gerçekler ve tek bir `Interval` nesnesine bağlıdır; `Interval` (`start`, `end`, `length`: String; `calculateLength`, `calculateEnd`) `Observable`'dan türer.
  - Observer yerine Command kalıbı da kullanılabilir. Biraz ek iş getirir, ama yapılmazsa katmanlaşma bozulur ve GUI sınıfı hem long method hem large class olur.

Hocanın Özellikle Vurguladığı Kısımlar

- Kod kusurlarının ve refactoring eylemlerinin İngilizce özgün adları bilinmeli
  - Hoca bu adların akılda kalması gerektiğini, adların da anlamlarını iyi anlattığını söyledi.
- Refactoring testleri değiştirmemeli
  - Refactoring dış arayüzü değiştirmez; bunu "yaptığımız refactoring eylemleri bizi unit test kodlarımızı değiştirmek zorunda bırakmamalı" diye de ifade etti. Introduce Parameter Object bu kuralı sınıf düzeyinde çiğner ama sistem düzeyinde çiğnemez.
- Anlaşılabilirlik koda yedirilmeli
  - Yorum satırlarına gerek kalmayacak kadar anlaşılır kod yazılmalı; iyi adlandırılmış metotlar yorumun yerini alır.
- Daha çok metot, daha çok parça
  - Yönetilecek parça sayısı ve metot çağırma yükü artar; ama uyumun yükselmesi ve anlaşılabilirlik gibi kazanımlar karşısında "günün sonunda kârdayız".
- Başarım kaygısı ölçülerek değerlendirilmeli
  - Metot çağırma yükü genelde fark edilmez, çünkü darboğaz başka yerdedir. Ama aynı sorgu farklı bilgi sistemlerine ya da ağ üzerinden yapılıyorsa, ya da kayıt sayısı büyükse, profiler ile ölçülüp karar verilmeli.
- Tekrarlanan kod ilişkisiz sınıflardaysa tasarım yanlıştır
  - Sorumluluklar büyük olasılıkla doğru atanmamıştır.
- İş mantığı GUI sınıfında tutulmaz
  - Katmanlaşma korunmalı; veri bilgi düzeyi sınıfına taşınıp Observer ile eşgüdüm sağlanmalı.

Kısa Tekrar Notları

- Kod kusuru ile refactoring eylemi arasında M×N ilişki vardır.
- Duplicated Code → Extract Method, Pull Up Method, Template Method.
- Long Method → Extract Method, Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
- Long Parameter List → Introduce Parameter Object, Preserve Whole Object.
- Large Class → Extract Class, Extract Subclass, Duplicate Observed Data.
- Extract Method: kodun bir kısmını amacını anlatan adla yeni metoda çıkar.
- Pull Up Method: kardeş sınıflardaki aynı metodu üst sınıfa çek; farklı olanı üstte soyut tanımla.

- Decompose Conditional: karmaşık koşulu `notSummer(date)` gibi adlandırılmış metoda çıkar.
- Introduce Parameter Object: `(start, end)` → `DateRange`.
- Preserve Whole Object: `withinRange(low, high)` → `withinRange(daysTempRange())`; bağımlılık artabilir.
- Replace Temp with Query: `basePrice` değişkeni → `basePrice()` metodu; çağrı sayısı artar.
- Replace Method with Method Object: `Order.price()` → `new PriceCalculator(this).compute()`.
- Extract Class: `Person` → `Person` + `Telephone Number`.
- Extract Subclass: `Job Item` → `Labor Item` alt sınıfı.
- Duplicate Observed Data: `Interval Window` (GUI) + `Interval` (iş mantığı), Observer ile.
- System Complexity görünümü: genişlik = üye alan, yükseklik = metot, renk = kod satırı.

Detaylı Açıklamalar

- Extract Method'un ikinci örneği (kod ekrandan, 00:30:00):

```
/*** ÖNCE: ***/
void printOwing( ) {
  Enumeration e = _orders.elements();
  double outstanding = 0.0;

  // print banner
  System.out.println("******************");
  System.out.println("* Customer Owes *");
  System.out.println("******************");

  // calculate outstanding
  while (e.hasMoreElements()) {
    Order each = (Order) e.nextElement();
    outstanding += each.getAmount();
  }

  //print details
  System.out.println ("name:" + _name);
  System.out.println ("amount" + outstanding);
}

/*** SONRA: ***/
void printOwing( ) {
  printBanner();
  printDetails(getOutstanding());
}

void printBanner() {
  System.out.println("******************");
  System.out.println("* Customer Owes *");
  System.out.println("******************");
}

double getOutstanding() {
  double outstanding = 0.0;
  for (Order each : _orders)
    outstanding += each.getAmount();
  /* for-each ile döngü kısaldı! */
  return outstanding;
}

void printDetails (double outstanding) {
  System.out.println ("name:" + _name);
  System.out.println ("amount" + outstanding);
```

```
}
```

- Extract Method neden anlaşılabilirliği artırır? Hoca bir insanın bir seferde odaklanabileceği bilgi parçası sayısının "5 artı eksi 2" olduğunu söyledi: daha az yetenekli olanlar aynı anda üç, daha yetenekli olanlar yedi bileşeni kafasında tutabilir. Uzun bir kodu ana hatlarıyla alt işlere ayırıp yalnız ilgilenilen alt işe odaklanmak bu yüzden kolaylık sağlar.
  - Ders sonrası not: Bu konuda en çok bilinen çalışma George Miller'ın "7 artı eksi 2" bulgusudur; hocanın verdiği "5 artı eksi 2" büyük olasılıkla daha sonraki, sınırı daha düşük gösteren çalışmalara dayanıyor.
- Metot çağırma ek yükü: Hoca bunu "context switching overhead" diye andı ve güncel derleyicilerin, Java Sanal Makinesi'nin ve .NET Runtime'ın bu yükü önemli oranda azalttığını, bir iki milisaniyelik ek yükün sistemin geri kalanında hissedilmeyeceğini söyledi.
  - Ders sonrası not: Hoca "context switching" ve "1-2 milisaniye" dedi; burada kastedilen metot (fonksiyon) çağırma ek yüküdür. "Context switch" genelde işletim sisteminin süreç ya da iş parçacığı değiştirmesi için kullanılır. Basit bir metot çağrısının yükü günümüz donanımında büyük olasılıkla nanosaniye düzeyindedir ve derleyiciler satır içi açma (inlining) ile bunu çoğu zaman tamamen kaldırır.
- Decompose Conditional örneği (kod ekrandan, 00:50:00):

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;

/*** SONRA: ***/
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);
private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
    date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
    return quantity * _summerRate;
}
private double winterCharge(int quantity) {
    return quantity * _winterRate + _winterServiceCharge;
}
```

- Hoca bunun karmaşıklığı "halının altına süpürmek" gibi olduğunu, ama üstteki hesap mantığını anlaşılır kıldığını söyledi. Önerdiği `isSummer` sürümünün kodu derste yazılmadı. De Morgan kuralına göre `isSummer(date)`, `!date.before(SUMMER_START) && !date.after(SUMMER_END)` olur (ajan ekledi).
- Replace Temp with Query örneği (kod ekrandan, 01:24:30):

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}

/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

- Temel fiyat 1000'den büyükse %5, değilse %2 indirim yapılır. Sonraki halde `getPrice()` çağrıldığında `basePrice()` bir kez `getPrice()` içinden, bir kez de `discountFactor()` içinden, toplam iki kez çağrılır.
- Döngüde Replace Temp with Query (kod ekrandan, 01:31:30):

```
void printRecipt() { /*** ÖNCE: ***/
    int totalAmount = 0, thisAmount;
    for( Item item : items ) {
        thisAmount = item.getCharge( );
        System.out.println(item.getName()+" "+thisAmount);
        totalAmount += thisAmount;
} }

void printRecipt() { /*** SONRA: ***/
    int totalAmount = 0;
    for( Item item : items ) {
        System.out.println(item.getName()+" "+item.getCharge());
        totalAmount += item.getCharge( );
} }
```

- `thisAmount` yalnız bir kez atanır ve iki kez okunur, bu yüzden her yerde `item.getCharge()` yazılabilir. `totalAmount` ise kümülatif toplamın birikimcisi olduğu için kalmak zorundadır. n öğe için çağrı sayısı n'den $2n$'ye çıkar.
- Metrik eşikleri tablosu (ekrandan, 01:48:10; Lanza ve Marinescu'nun kitabı, Tablo 2.2):

	Java				C++			
Metric	Low	Average	High	VeryHigh	Low	Average	High	VeryHigh
WMC	5	14	31	47	4	23	72	108
AMW	1.1	2.0	3.1	4.7	1.0	2.5	4.8	7.0
LOC/Class	28	70	130	195	20	90	240	360
NOM/Class	4	7	10	15	4	9	15	23

- WMC (Weighted Methods per Class), sınıfın ağırlıklandırılmış metot sayısıdır; kitapta $WMC = \frac{CYCLO}{LOC} \cdot \frac{LOC}{Method} \cdot \frac{NOM}{Class}$ diye üç terime ayrılır (ekrandan, 01:48:10). AMW ortalama metot ağırlığı (CYCLO/Method), LOC/Class sınıf başına kod satırı, NOM/Class sınıf başına metot sayısıdır. Hoca ayrıntıya girmedi.
- Duplicate Observed Data'nın sonraki yapısı (ekrandan, 02:00:00):

```
Interval Window                          <<interface>> Observer
  startField, endField, lengthField: TextField   - - -|> (gerçekler)
  StartField_FocusLost, EndField_FocusLost, LengthField_FocusLost
        |
        | 1
        v
Interval                                 Observable
  start, end, length: String             ------|> (kalıtım)
  calculateLength, calculateEnd
```

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.